

UNIT II

ARRAYS

Suppose we wish to arrange the percentage marks obtained by 100 students in ascending order. In such case we have two options Construct 100 variables to store 100 percentage marks, construct one variable (called an array) capable of storing or holding all the 100 values. Obviously the second option is better. Moreover there are certain logics that cannot be dealt with, without the use of arrays.

To process large amounts of data, C supports a derived data type known as “**ARRAYS**” that can be used for such applications.

Definition:

An Array is a fixed size of sequenced collection of elements of same data type.

- An Array is a collection of homogeneous data items that share a common name.
- Arrays are used to store more than one value in a single variable.

TYPES OF ARRAYS

There are 3 types of arrays

- One-dimensional Array
- Two- dimensional Array
- Multi- dimensional Array

ONE-DIMENSIONAL ARRAY:

An Array with single subscript is called one dimensional Array (or) single subscripted array.

Syntax: datatype array_name [size];

Ex: int a[5];

a[0]	a[1]	a[2]	a[3]	a[4]
0	1	2	3	4

We can store (or) access each array element using its index.

- An array index starts from zero.
- The array elements are stored in continuous memory locations.

INITIALIZATION:

An array can be initialized at two stages

- Compile time
- Run time

Compile Time Initialization:

Syntax: datatype array_name [size] = { list of values } ;

Ex: int a[3] = { 10, 20, 30 };

a	10	20	30
	0	1	2

- If only values are initialized then the remaining values will be automatically zero.

Ex: `int a[4] = { 10,20 };`

a	10	20	0	0
	0	1	2	3

- While initializing all the values we can omit the size of an array.

Ex: `int a[ ] = { 1, 3, 5, 7};`

a	1	3	5	7
	0	1	2	3

- The more values are initialized then the max size of an array, then the compiler shows an error.

Ex: `int a[3] = { 1, 3, 5, 7};` ❌

Run time initialization:

- This is used for large size of arrays

Ex: Runtime Initialization
`int a[100], i;`
`for (i=0;i<=100;i++)`
`a[i]=0;`

Ex: Compiletime Initialization
`int a[100] = { 0 };`

- In C, the name of an array represents the address of the first element(base address)
- C calculates the address of each array element using the formulae.

Element address = base address + index * Scale factor

Scale factor means size of data type (In bytes) size of the element.

Ex: `int a[5] = { 10, 20, 30, 40, 50};`

	a[0]	a[1]	a[2]	a[3]	a[4]
a	10	20	30	40	50
	1000	1002	1004	1006	1008

Address of a[3] = Base Address of a + index * Scale Factor
 = 1000 + 3 * 2
 = 1006

/*PROGRAM TO READ AND PRINT A 1DIM ARRAY*/

`#include<stdio.h>`

`#include<conio.h>`

`void main()`

`{`

`int a[5],i;`

`clrscr();`

`for(i=0;i<5;i++)`

`{`

`printf("\n\t Enter a%d=",i);`

`scanf("%d",&a[i]);`

`}`

`printf("\nArray entered is:");`

```

    for(i=0;i<5;i++)
    {
        printf("\ta[%d]=%d",i,a[i]);
    }
    getch();
}

```

o/p:

Enter a0=1

Enter a1=2

Enter a2=3

Enter a3=4

Enter a4=5

Array entered is:

a[0]=1 a[1]=2 a[2]=3 a[3]=4 a[4]=5

/*PROGRAM TO READ AND PRINT THE NUMBERS AND THEIR SUM USING ARRAYS*/

```

void main()
{
    int a[20],i,n,sum=0;
    clrscr();
    printf("\nHow many no.s you want to sum:");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\n\t Enter a[%d]= ",i);
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
    {
        printf("\n\t a[%d]=%d",i,a[i]);
        sum+=a[i];
    }
    printf("\n\t sum= %d",sum);
    getch();
}

```

o/p:

How many no.s you want to sum:5

Enter a[0]= 2

Enter a[1]= 5

Enter a[2]= 3

Enter a[3]= 1

Enter a[4]= 8

a[0]=2 a[1]=5 a[2]=3 a[3]=1 a[4]=8 sum= 19

```
/*.PROGRAM TO PRINT THE ODD SUM AND EVEN SUM OF N ARRAY ELEMENTS WHERE  
N IS SUPPLIED BY THE USER.*/
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int a[50],i,n,esum=0,osum=0;
```

```
    clrscr();
```

```
    printf("\n How many elements you want to enter?");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        printf("\n\tEnter a%d=",i);
```

```
        scanf("%d",&a[i]);
```

```
        if((a[i]%2)==0)
```

```
            esum+=a[i];
```

```
        else
```

```
            osum+=a[i];
```

```
    }
```

```
    printf("\n The even sum and odd sum are: %d & %d", esum, osum);
```

```
    getch();
```

```
}
```

o/p:

How many elements you want to enter? 5

Enter a0=2

Enter a1=3

Enter a2=4

Enter a3=5

Enter a4=6

The even sum and odd sum are: 12 & 8

```
/*PROGRAM TO FIND MIN AND MAX USING ARRAYS*/
```

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int a[5],i,min,max;
```

```
    for(i=0;i<5;i++)
```

```
    {
```

```
        printf("\n\tEnter a%d= ",i);
```

```
        scanf("%d",&a[i]);
```

```
    }
```

```
    min=a[0];
```

```
    max=a[0];
```

```
    for(i=0;i<5;i++)
```

```
    {
```

```
        printf("\n\ta%d=%d",i,a[i]);
```

```
        if(min>a[i])
```

```

        min=a[i];
        if(max<a[i])
            max=a[i];
    }

    printf("\n\n\t min=%d\tmax=%d",min,max);
    getch();
}
O/P

```

Enter a0= 6
 Enter a1= 3
 Enter a2= 0
 Enter a3= 2
 Enter a4= 9

a0=6 a1=3 a2=0 a3=2 a4=9
 min = 0 max=9

TWO DIMENSIONAL ARRAYS:

An array with two subscripts is called as 2-Dim array or double subscripted array. These are used to store a table of values.

Syntax to declare 2-D arrays

data type array name [row size][column size];

Ex: int a[2][3];

Creates an array with 2 rows and 3 columns

	0	1	2
0			
1			

a

- We can access each individual element using its row number and column number
- Ex: a[0][1] represents the value in 0th row and 1st column.

Initialization:

Ex: int a[2][3] = { 1, 2, 3, 4, 5, 6 };

This Statement assigns first 3 elements to first row and next 3 elements to second row

a[0][0] = 1 a[0][1] = 2 a[0][2] = 3
 a[1][0] = 4 a[1][1] = 5 a[1][2] = 6

	0	1	2
0	1	2	3
1	4	5	6

a

- Initialization will be done by row by row.
- When an array is completely initialized with all the elements then there is no need to specify size in the 1st subscript.

Ex: int a[][3]={ { 1, 2, 3 }, { 4, 5, 6 } };

- If some values are not given, then they will be zero

Ex: `int a[2][3] = { { 1, 2 }, { 4 } };`

	0	1	2
0	1	2	0
1	4	0	0

a

- We can initialize all the elements as zero,

Ex: `int a[2][3] = { 0 };`

/*PROGRAM TO READ AND PRINT THE ELEMENTS OF A 2 DIM ARRAY (3X3 MATRIX) */

```
void main()
{
    int a[3][3],i,j;
    clrscr();
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("\n\t Enter a%d%d=",i,j);
            scanf("%d",&a[i][j]);
        }
    }
    for(i=0;i<3;i++)
    {
        for(j=0;j<3;j++)
        {
            printf("\n\n\t a%d%d=%d",i,j,a[i][j]);
        }
    }
    getch();
}
```

o/p:

```
Enter a00=1
Enter a01=2
Enter a02=3
Enter a10=4
Enter a11=5
Enter a12=6
Enter a20=7
Enter a21=8
Enter a22=9

a00=1  a01=2  a02=3
a10=4  a11=5  a12=6
a20=7  a21=8  a22=9
```

/*PROGRAM TO ADD AND PRINT SUM OF TWO MATRICES USING ARRAYS */

```
#include<stdio.h>
#include<conio.h>
void main()
{
```

```

int a[3][3],b[3][3],c[3][3],i,j;
clrscr();

for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        printf("\tEnter a%d%d=",i,j);
        scanf("%d",&a[i][j]);
    }
}

for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        printf("\t Enter b%d%d=",i,j);
        scanf("%d",&b[i][j]);
    }
}

for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        c[i][j]=a[i][j]+b[i][j];
        printf("\t%d",c[i][j]);
    }
}
getch();

```

}

o/p:

Enter a00=1	Enter a21=8	Enter b12=4
Enter a01=2	Enter a22=9	Enter b20=3
Enter a02=3	Enter b00=9	Enter b21=2
Enter a10=4	Enter b01=8	Enter b22=1
Enter a11=5	Enter b02=7	10 10 10
Enter a12=6	Enter b10=6	10 10 10
Enter a20=7	Enter b11=5	10 10 10

/*PROGRAM TO PRINT THE PRODUCT OF TWO 3X3 MATRICES USING ARRAYS*/

#include<stdio.h>

#include<conio.h>

void main()

{

int a[3][3],b[3][3],c[3][3],i,j,k;

clrscr();

for(i=0;i<3;i++)

{

for(j=0;j<3;j++)

{

printf("\tEnter a%d%d=",i,j);

scanf("%d",&a[i][j]);

}

}

for(i=0;i<3;i++)

{

for(j=0;j<3;j++)

{

printf("\t Enter b%d%d=",i,j);

scanf("%d",&b[i][j]);

}

}

/*

for(i=0;i<3;i++)

{

for(j=0;j<3;j++)

{

printf("\n\ta%d%d=%d",i,j,a[i][j]);

}

}

for(i=0;i<3;i++)

{

for(j=0;j<3;j++)

{

printf("\n\t b%d%d=%d ",i,j,b[i][j]);

}

}

Here code within the comments prints the a[3][3] b[3][3] matrices once.* /

for(i=0;i<3;i++)

{

for(j=0;j<3;j++)

{

c[i][j]=0;

for(k=0;k<3;k++)

{

c[i][j]=c[i][j]+a[i][k]*b[k][j];

```

        }
        printf("\t%d",c[i][j]);
    }
    printf("\n");
}
/* for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    {
        printf("\t%d",c[i][j]);
    }
}
*/
getch();
}

```

o/p:

Enter a00=1
 Enter a01=2
 Enter a02=3
 Enter a10=4
 Enter a11=5
 Enter a12=6
 Enter a20=7
 Enter a21=8
 Enter a22=9

Enter b00=1
 Enter b01=0
 Enter b02=0
 Enter b10=0
 Enter b11=1
 Enter b12=0
 Enter b20=0
 Enter b21=0
 Enter b22=1

Product of the above two matrices is

1	2	3
4	5	6
7	8	9

(Matrix b[3][3] is Unit Matrix)

/*PROGRAM TO SORT THE ELEMENTS OF AN ARRAY IN ASCENDING ORDER*/

```

void main()
{
    int a[10],i,n,j,t,k;
    clrscr();
    printf("\nHow many no.s you enter?");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter element %d: ",i+1);
        scanf("%d",&a[i]);
    }
    for(i=0;i<n;i++)
    {
        for(j=0;j<n-i-1;j++)

```

```

        {
            if(a[j]>a[j+1])
            {
                t=a[j];
                a[j]=a[j+1];
                a[j+1]=t;
            }
        }
    }
    printf("\n Sorted elets are: \n");
    for(i=0;i<n;i++)
        printf("\t%d",a[i]);
    getch();
}

```

o/p:

How many no.s you enter? 5

Enter element1: 99

Enter element2: 34

Enter element3: 21

Enter element4:44

Enter element5:12

Sorted elets are:

12 21 34 44 99

Note that the above program is Just logic of Bubble sort which you will learn in Unit 7. As a token of application of Arrays I mentioned here

MULTI DIMENSIONAL ARRAYS

An array with more than two dim is called as multi dim array.

Syntax:

datatype array name [index] [index] [index].....;

Syntax to declare a 3-d array

datatype array_name[planes][rows][cols];

Ex: int a[3][2][3];

The first subscript represents the number of tables or planes to be created.

The second and third subscript represents the number Of rows and cols for each plane.

Initialization:

int a[3][2][3]

= { { {1, 2, 3}, {4, 5, 6} }, { {7, 8, 9}, {10, 11, 12} }, { {13, 14, 15}, {16, 17, 18} } };

	0	1	2
0	1 a[0][0][0]	2 a[0][0][1]	3 a[0][0][2]
1	4 a[0][1][0]	5 a[0][1][1]	6 a[0][1][2]

a[0] plane

	0	1	2
0	7 a[1][0][0]	8 a[1][0][1]	9 a[1][0][2]
1	10 a[1][1][0]	11 a[1][1][1]	12 a[1][1][2]

a[1] plane

	0	1	2
0	13 a[2][0][0]	14 a[2][0][1]	15 a[2][0][2]
1	16 a[2][1][0]	17 a[2][1][1]	18 a[2][1][2]

a[2] plane

/* PROGRAM TO READ AND PRINT A 3-D ARRAY */

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int a[2][2][2],i,j,k;
```

```
    clrscr();
```

```
    for(i=0;i<2;i++)
```

```
    {
```

```
        for(j=0;j<2;j++)
```

```
        {
```

```
            for(k=0;k<2;k++)
```

```
            {
```

```
                printf("\n\t enter a[%d][%d][%d]=",i,j,k);
```

```
                scanf("%d",&a[i][j][k]);
```

```
            }
```

```
        }
```

```
}
```

```

    }
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            for(k=0;k<2;k++)
            {
                printf("\n\t a[%d][%d][%d]=%d",i,j,k,a[i][j][k]);
            }
        }
    }
    getch();
}
o/p:

```

```

enter a[0][0][0]=1
enter a[0][0][1]=2
enter a[0][1][0]=3
enter a[0][1][1]=4
enter a[1][0][0]=5
enter a[1][0][1]=6
enter a[1][1][0]=7
enter a[1][1][1]=8

```

```

a[0][0][0]=1
a[0][0][1]=2
a[0][1][0]=3
a[0][1][1]=4
a[1][0][0]=5
a[1][0][1]=6
a[1][1][0]=7
a[1][1][1]=8

```

APPLICATIONS OF ARRAYS:

- ✓ Searching
- ✓ Sorting
- ✓ Matrix applications
- ✓ String manipulations
- ✓ Function parameters
- ✓ Structures
- ✓ Pointers
- ✓ Data structures
- ✓
- ✓

✓ STRINGS

“A string is a sequence of characters which is treated as a single data item.”

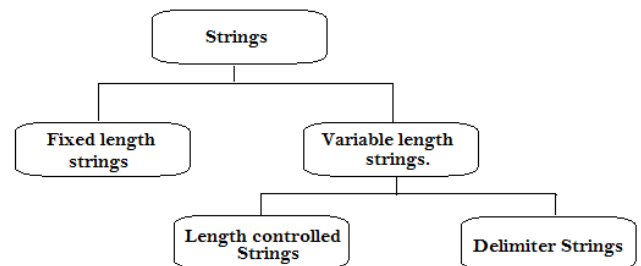
There are two types of strings.

1. Fixed length strings
2. Variable length strings.

1). Fixed length strings:

These are the strings with fixed size. The main problem in this type is deciding size of a string.

The fixed length strings must end with special character such as space.



2). Variable Length Strings:

These are strings with variable size. If we want to store a single character, we can provide a single memory location. If it is needed to store 30 characters, we can expand it to store all the 30 characters.

The end of strings can be determined using 2 techniques.

1. Length controlled strings
2. Delimiter strings

1) Length controlled strings:

These strings add a count that specifies the number of characters in a string. This specifies the number of characters in a string. This count is later used by the string manipulation functions to determine the size of string.

EX:

7	P	R	O	G	R	A	M
---	---	---	---	---	---	---	---

2) Delimiter strings:

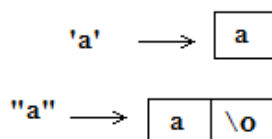
Delimiter strings places a null character ('\0') at the end of a string, which indicates the end of a string

Ex:

P	R	O	G	R	A	M	'\0'
---	---	---	---	---	---	---	------

C STRINGS:

- A String in c is an array of characters which is terminated by a null character.
- As the string stored in an array, the string name is a pointer to the string.
- The difference between storage of a single character and a character string in memory is, a single character requires only one memory location, where as a character string requires two memory locations, one for character and another for null character.



Need for null character

- String is not a datatype in c, but it is considered as a data structure stored in an array.
- A string is of variable length stored in a fixed size array.
- Sometimes the array size will be more than the string size. In such cases to represent

the end of a string, a null character will be used. Otherwise a string will be displayed along with the garbage.

Declaring and Initializing Strings:

Syntax to declare a string:

```
char arrayname[size];
```

The size represents the maximum number of characters to be stored in an array.

Ex: `char s[10];`

- In "s", we can store 9 characters and 10th character will be the null character.
- The null character will be assigned automatically by the compiler after taking the input.
- So the size of a string must be the number of characters to be stored +1.

INITIALIZATION:

Like numeric arrays, we can initialize a character array.

- Ex: `char city [10]="HYDERABAD";`
(Or)
`char city [10]={'N','E','W',' ','D','E','L','H','I'};`
- In some of the cases (where the string value is initialized at declaration), there is no need to specify the size of the string.
Ex: `char s[]="ECE";` (String s will be declared with size 4)
- Ex: `char s[9]= "VIGNAN"`
This statement creates an array of size 9 and puts the string in it and terminated by a null character.
- Ex: `char s[2] = "RAVI";` → invalid, Because the string length is more than the size of 2.
- We can't separate declaration and initialization of a string.
Ex: `char s[5];`
`s="GOOD";` → invalid.
- We can't assign a string to another string variable.
Ex: `char ch1[5]="DOIT";`
`char ch2[5];`
`ch2=ch1;` → is not allowed

/*PROGRAM TO PRINT A STRING */

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char s1[10]="HYDERABAD";
    char s2[10]="NEW DELHI";
    char s3[15]={'R','A','V','I',' ','&','C','P','D','S'};
    clrscr();
    printf("\nString1: %s\n String2: %s \n String3: %s",s1,s2,s3);

    getch();
}
```

O/P:

String1: HYDERABAD

String2: NEW DELHI

String3: RAVI &CPDS

STRING INPUT/OUTPUT FUNCTIONS:

There are two ways to read and print strings

1. Formatted input/output functions.
2. Special string functions.

FORMATTED INPUT/OUTPUT FUNCTIONS:

The input function to read a string is scanf with %s format specifier.

- Don't give space while entering the Name.
 - For reading a string no need to specify "&" before the variable in scanf function.
 - The problem with scanf is, it terminates the input when a white space is found.
 - A white space includes tabs, blank space, next line (enter, \n)
-
- We can specify the field width of a string i.e., the number of characters to read.

Ex: char name[7];

scanf("%5s", &name);

- If the input is: "Hai", then the string will be stored in the following form

Hai →

H	a	i	\0			
0	1	2	3	4	5	6

- if the i/p is "ABCDEFGH", then the string will be stored as

"ABCDEFGH" →

A	B	C	D	E	\0	
0	1	2	3	4	5	6

/*PROGRAM TO PRINT A STRING WITH FORMATS*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
```

```
void main()
{
    char s1[10]="HYDERABAD";
    char s2[15];
    clrscr();
    printf("\nEnter String3: ");
    scanf("%5s",s2);
    printf("\n|%20.10s|\n|%-20.10s|\n|%20.5s|",s1,s1,s2);
    getch();
}
```

O/P:

```
Enter String3: Krishna
|      HYDERABAD|
|HYDERABAD      |
|      Krish|
```

GETCHAR, PUTCHAR FUNCTIONS:

get char ():

getchar for is used t read a character

Syntax: variable = getchar();

put char ():

functin i sused to print a single character as o/p.

syntax: putchar (variable)

Ex:

```
main()
{
    char c;
    c = getchar();
    putchar(c)
}
```

/*PROGRAM TO READ A LINE OF TEXT*/

```
#include<stdio.h>
#include<conio.h>
void main()
{
```

```
    char ch,line[40];
    int i=0;
    clrscr();
    printf("\nEnter Text\n\t");
```

```

do
{
    ch=getchar();
    line[i]=ch;
    i++;
}while(ch!='\n');
i--;
line[i]='\0';
printf("\n\n%s",line);
getch();
}

```

O/P:

Enter Text

Progrm Read Lineof Text

Progrm Read Lineof Text

Note:

getch() expects a character through the keyboard which is not visible, where as

getche() expects a character through the keyboard which is visible.

getchar() expects a character through the keyboard which is visible and stores that character in a char variable.

Special string functions:

There are two special functions for strings.

1. puts
2. gets

gets():

It is used to read a line of text including white spaces.

syntax: gets(variable);

gets() function is also called as line to string i/p function.

puts():

It is used to print the given line of text on the console.

Syntax: puts(variable)

puts() function is also called as line to string o/p function.

/*PROGRAM TO READ PRINT STRINGS USING PUTS AND GETS FUNCTIONS*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```

{
    char line[40];
    clrscr();
    printf("\nEnter Text\n");
    gets(line);
    printf("\nYou Entered\n");
    puts(line);
}

```

```
    getch();  
}
```

O/P:

Enter Text

Puts & Gets Functions always recommended

You Entered

Puts & Gets Functions always recommended

Predefined/Built-in string functions / String Handling Functions:

- | | |
|--------------|---------------------------|
| 1. strlen() | 9. strrchr() |
| 2. strcpy() | 10. strstr() |
| 3. strncpy() | 11. strpbrk() |
| 4. strcat() | 12. strrev() |
| 5. strncat() | 13.strupr() |
| 6. strcmp() | 14. strlwr() |
| 7. strncmp() | 15. tolower() //char fun |
| 8. strchr() | 16. toupper() // char fun |

[Type text]

1. strlen():

The string length function is used to find the no of chrs ina given string excluding null character.

syn: var = strlen(strng);

/*PROGRAM TO ILLUSTRATE ABOUT STRING FUNCTION STRLEN*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char s1[20];
    clrscr();
    printf("\nEnter Some Text: ");
    gets(s1);
    printf("\nString Length=%d",strlen(s1));
    getch();
}
```

o/p:

Enter Some Text: This is a String.

String Length=17

2. strcpy(): The string copy function strcpy is used to copy the contents of one string to another string.

syn: strcpy(string2, string1);

Contents of S1 will be copied to S2.

/* PROGRAM TO COPY THE ONE STRING INTO ANOTHER*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char s1[30],s2[30];
    clrscr();
    printf("\nEnter String1:");
    gets(s1);
    strcpy(s2,s1);
    printf("\nString2=%s",s2);

    getch();
}
```

O/P:

Enter String1: This String Will Be Copied

String2=This String Will Be Copied

3. strncpy():

The string number copy function strncpy() is used to copy specific number of characters from one string to another string .

syn: strncpy(s2,s1,n).

It copies n characters from s1 to s2

/*PROGRAM TO COPY N CHARS OF ONE STRING INTO ANOTHER STRNCPY*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s1[20],s2[20];
```

```
    clrscr();
```

```
    printf("\nEnter String1: ");
```

```
    scanf("%s",s1);
```

```
    strncpy(s2,s1,6);
```

```
    printf("\nString2=%s",s2);
```

```
    getch();
```

```
}
```

O/P:

Enter String1: This is String.

String2=This i

4) strcat():

The string concatenate function is used to join two strings together.

Syntax:: strcat(string 1,string 2);

The string 2 will be appended at the end of string 1.

/*PROGRAM TO CONCATANATEF ONE STRING INTO ANOTHER STRCAT*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s1[20],s2[20];
```

```
    clrscr();
```

```
    printf("\nEnter String1:");
```

```
    gets(s1);
```

```
    printf("\nEnter String2:");
```

```
    gets(s2);
```

```
    strcat(s1,s2);
```

```
    printf("\nStrings are=%s",s1);
```

```
    getch();
```

```
}
```

[Type text]

O/P:

Enter String1:New Delhi

Enter String2:Hyderabad

Strings are=New DelhiHyderabad

5) strncat():

The string number concatenate function strncat is used to concatenate the specified number of characters.

syntax:: strncat(string 1,string 2,n);

The first n chrs from string appended to string 1.

/*PROGRAM TO CONCATANATE ONE STRING INTO ANOTHER STRNCAT*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s1[20],s2[20];
```

```
    clrscr();
```

```
    printf("\nEnter String1:");
```

```
    gets(s1);
```

```
    printf("\nEnter String2:");
```

```
    gets(s2);
```

```
    strncat(s1,s2,5);
```

```
    printf("\nStrings are=%s",s1);
```

```
    getch();
```

```
}
```

O/P:

Enter String1:Hyderabad

Enter String2:NewDelhi

Strings are=HyderabadNewDe

6) strcmp():

String compare function is used to compare the given two strings.

This function return 0 if both are equal; else it returns the difference between the ASCII values of the first nonmatching characters.

Syntax: strcmp(string 1,string 2);

/*PROGRAM TO COMPARE TWO STRINGS STRCMP*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

[Type text]

```
char s1[20],s2[20];
int n;
clrscr();
printf("\nEnter String1:");
gets(s1);
printf("\nEnter String2:");
gets(s2);
n= strcmp(s1,s2);

if(n==0)
    printf("\nStrings are Same");
else
    if(n<0)
        printf("\n String1 < String2");
    else
        printf("\n String1 > String2");
getch();
}
```

O/P:

Enter String1:RAVI

Enter String2:ravi

String1 < String2

7) strncmp():

The string number compare function strncmp is used to compare only the specified number of characters in given string.

syntax:: strcmp(string 1,string 2,n);

/*PROGRAM TO COMPARE TWO STRINGS STRNCMP UPTO N CHARS IN STRING1*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s1[20],s2[20];
```

```
    int n;
```

```
    clrscr();
```

```
    printf("\nEnter String1:");
```

```
    gets(s1);
```

```
    printf("\nEnter String2:");
```

```
    gets(s2);
```

```
    n= strncmp(s1,s2,5);
```

```
    if(n==0)
```

```
        printf("\nStrings are Same up to 5 Characters");
```

```
    else
```

```
        if(n<0)
```

[Type text]

```
                printf("\n String1 < String2");
            else
                printf("\n String1 > String2");
        getch();
    }
```

O/P:

Enter String1:RaviKrishna

Enter String2:RaviKishore

Strings are Same up to 5 Characters

8) strchr():

The string character function strchr is used to find the first occurrence of the given character in the given string.

If a character is present in the string ,it returns the address of that character ,else returns a null pointer.

Syntax: pntvariable = strchr(string,'character');

/*PROGRAM TO FIND THE POSITION OF CHAR IN THE STRING - STRCHR*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s[20];
```

```
    char *p;
```

```
    clrscr();
```

```
    printf("\nEnter String1:");
```

```
    gets(s);
```

```
    p= '\0';
```

```
    p = strchr(s,'R');
```

```
    if(*p==0)
```

```
        printf("\n char R does not exist in the String");
```

```
    else
```

```
        printf("\n 'R' found in %d position",p-s+1);
```

```
    getch();
```

```
}
```

O/P1:

Enter String1:HYDERABAD

'R' found in 5 position

O/P2:

Enter String1:NEW DELHI

char R does not exist in the String

[Type text]

9) strrchr():

The string rear character function is used to find the occurrence of the given character in the given string.

/*PROGRAM TO FIND THE POSITION OF CHAR IN THE STRING IN REVERSE - STRRCHR*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char s[20];
    char *p;
    clrscr();
    printf("\nEnter String1:");
    gets(s);
    p= '\0';
    p = strrchr(s,'N');

    if(*p==0)
        printf("\n char N does not exist in the String");
    else
        printf("\n 'N' found in %d position in Reverse",p-s+1);
    getch();
}
```

O/P:

Enter String1:Hyderabad

char N does not exist in the String

Enter String1:VIGNAN

'N' found in 6 position in Reverse

10) strstr():

The string substring function is used to find whether the given substring exists in the given string or not.

If the substring is found in the string it returns the address of the (sub)string ,else returns a null.

Syntax: strstr (string, substring)

/*PROGRAM TO FIND THE SUBSTRING IN A STRING - STRSTR*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char s[20]="This is a String", sub[10]="s is";
    char *p;
    clrscr();
```

[Type text]

```
printf("\nYour String is: %s\n Sub String is: %s",s,sub);
```

```
p = strstr(s,sub);
```

```
printf("\nstrstr function returns the string: %s",p);
```

```
getch();
```

```
}
```

O/P:

Your String is: This is a String

Sub String is: s is

strstr function returns the string:

s is a String

11) strpbrk():

The string pointer break function returns a pointer of a substring in the given string.

Syntax: strpbrk(string , substring).

/*PROGRAM FOR POINTER BREAK FUNCTION IN C - STRPBRK*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s[20]="This is a String",sub[10]="surya";
```

```
    char *p;
```

```
    clrscr();
```

```
    printf("\nYour String is: %s\n Sub String is: %s",s,sub);
```

```
    p = strpbrk(s,sub);
```

```
    printf("\nstrpbrk function returns the string: %s",p);
```

```
    getch();
```

```
}
```

O/P:

Your String is: This is a String

Sub String is: surya

strpbrk function returns the string:

s is a String

12) strrev():

This is used to reverse the string.

/*PROGRAM TO ILLUSTRATE ABOUT STRING FUNCTION STRREV*/

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char s[40]="This is a String";
```

```
    clrscr();
```

[Type text]

```
printf("\nString=%s",s);
printf("\n\nString In Reverse is =%s",strrev(s));
getch();
}
```

String= This is a String

String In Reverse is = gnirtS a si sihT

13) strlwr():

The string lower function strlwr is used to convert the upper case letters to lower case.

14)strupr():

The string upper functionstrupr is used to convert the lower case letters to upper case.

/* PROGRAM TO PRINT THE STRING IN UPPER CASE - STRUPR*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char s[20]="This is a String";

    clrscr();
    printf("\nYour String is: \n%s",s);
    printf("\nYour String in Upper case is:\n %s",strupr(s));
    printf("\nYour String in Lower case is:\n %s",strlwr(s));
    getch();
}
```

O/P:

Your String is:

rAvI kRiShNa

Your String in Upper case is:

RAVI KRISHNA

Your String in Lower case is:

ravi krishna

15) tolower():

This function is used to convert a single character into lower case.

16) toupper():

This function is used to convert a single character into upper case.

**/*PROGRAM TO PRINT THE STRING IN UPPERCASE AND LOWER CASE USING
TOUPPER & TOLOWER FUNCTIONS */**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
```

[Type text]

```
void main()
{
    int i=0;
    char ch, s[20]="rAvI kRiShNa";

    clrscr();

    printf("\nYour String is: %s",s);
    printf("\nYour String in Upper case is: ");

    while(s[i]!='\0')
    {
        ch=toupper(s[i]);
        putchar(ch);
        i++;
    }
    i=0;

    printf("\nYour String in Lower case is: %s ");

    while(s[i]!='\0')
    {
        ch=tolower(s[i]);
        putchar(ch);
        i++;
    }
    getch();
}
```

O/P:

Your String is: rAvI kRiShNa

Your String in Upper case is: RAVI KRISHNA

Your String in Lower case is: ravi krishna

Array of Strings:

We can store a set of strings in a single variable by using 2D character array.

Syntax: char array[No. Of strings] [size of each string]

- In this we can store 5(names) strings, each of size 10.
- We can use 1D array syntax to store or access the strings.

*******Write a program to sort the names in the alphabetical order.**

**/*PROGRAM TO ILLUSTRATE ABOUT ARRAY OF STRINGS =
SORTING STRINGS IN DICTIONARY ORDER*/**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
```

[Type text]

```
{
    char s[5][10],t[10];
    int i,j;
    clrscr();
    for(i=0;i<5;i++)
    {
        printf("\n Enter name%d",i+1);
        gets(s[i]);
    }

    for(i=0;i<5;i++)
    {
        for(j=0;j<5;j++)
        {
            if( strcmp(s[i],s[j])<0)
            {
                strcpy(t,s[i]);
                strcpy(s[i],s[j]);
                strcpy(s[j],t);
            }
        }
    }
    for(i=0;i<5;i++)
    {
        printf("\n%d\t",i+1);
        puts(s[i]);
    }
    getch();
}
```

O/P:

Enter name1RAMA
Enter name2REHMAN
Enter name3REMO
Enter name4REONE
Enter name5RAVI

1	RAMA
2	RAVI
3	REHMAN
4	REMO
5	REONE

[Type text]

Additional Programs

/*PROGRAM TO FIND THE LENGTH OF A STRING WITHOUT USING STRING HANDLING FUNCTIONS*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    int i=0;
    char s1[20];
    clrscr();
    printf("\nEnter String1: ");
    gets(s1);
    while((s1[i])!='\0')
        i++;
    printf("\n Length of String1=%d",i);
    getch();
}
```

Enter String1: Vignan@Hyderabad

Length of String1=16

=====

/* PROGRAM TO COPY THE CONTENTS OF ONE STRING INTO ANOTHER WITHOUT STRING HANDLING FUNCTIONS*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    int i=0;
    char c,s1[20];
    char s2[20];
    clrscr();
    printf("\nEnter String: ");
    gets(s1);
    while((c=s1[i])!='\0')
    {
        s2[i]=c;
        i++;
    }
    s2[i]='\0';
    printf("\nCopied String=%s",s2);
    getch();
}
```

O/P:

[Type text]

Enter String: Hyderabad Vignan
Copied String=Hyderabad Vignan

=====

=

**/* PROGRAM TO REVERSE THE STRING WITHOUT STRING HANDLING
FUNCTIONS*/**

#include<stdio.h>

#include<conio.h>

#include<string.h>

void main()

{

int i=0,j=0;

char s1[20],s2[20];

clrscr();

printf("\nEnter String1: ");

gets(s1);

while((s1[i])!='\0')

i++;

i--;

for(;i>=0;i--)

{

s2[j]=s1[i];

j++;

}

s2[j]='\0';

printf("\n String1=%s \n Reverse String2=%s",s1,s2);

getch();

}

O/P:

Enter String1: Ravi Krishna

String1=Ravi Krishna

Reverse String2=anhSirK iVaR

Array of Pointers:

One of the important use of pointers is handling a table of strings.

Example: `char s[3][10];`

In s, we can store 3 strings of same size, each of 10 characters.

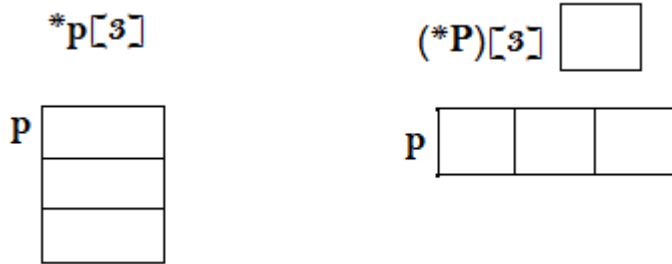
For this we can declare an array of type characters .

Example: `char *&s={"program", "RAVI", "GOOD"}`

The array of strings of varying length is called "**RAGGED ARRAYS**".

[Type text]

The difference between (*p)[3] is, p[3] is an array of 3 pointers (*p)[3] is a pointer pointing to 3 elements.



/* PROGRAM TO ILLUSTRATE ABOUT ARRAY OF POINTERS*/

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char *s[5];
    int i;
    clrscr();
    for(i=0;i<5;i++)
    {
        printf("\n Enter name%d",i+1);
        gets(s[i]);
    }
    for(i=0;i<5;i++)
    {
        printf("\n%d\t",i+1);
        puts(s[i]);
    }
}
```

O/P:

```
Enter name1VIGNAN INSTITUTE
Enter name2OF TECHNOLOGY &
Enter name3SCIENCE
Enter name4COMPUTER
Enter name5SCIENCE DEPT
1    VIGNAN INSTITUTE
2    OF TECHNOLOGY &
3    SCIENCE
4    COMPUTER
5    SCIENCE DEPT
```

Pointers And Character strings:

We can also declare strings to pointer variables after declaring it

Example: char *s;

s="prog";

[Type text]

This type of assignment is not valid for character arrays.

/* PROGRAM TO FIND THE LENGTH OF A STRING USING POINTERS*/

```
main()
{
    char *p,*n;
    p="prog";
    n=p;
    clrscr();
    printf("%d%s",strlen(n),n);
    getch();
}
```

O/P: 4 prog

STRUCTURES

Structure:

“A Structure is a collection of elements of different data type.”

We can store values of different types using a single name is STRUCTURE

DECLARATION:

We can declare a structure in two ways

1. Tagged structure
2. type defined structure

1.Tagged structure:

Syntax:

Struct tagname

```
{
    datatype memeber1;
    datatype memeber2;
    ---
};
```

- Tagged structure begins with keyword “struct”.
- The tag name is the name of the structure.
- The variables declared be terminated with a semi colon “;”.
- A structure will always be terminated with “;”.

Ex: struct student

```
{
    char sno[10];
    char sname[20];
    int m1,m2,m3;
    float avg;
```

[Type text]

};

2. typedef structures:

A structure declared using the keyword “typedef” is called type defined structure.

Syntax:

```
typedef struct
{
    datatype member1;
    datatype member2;
    - - -
}tagname;
```

Ex: typedef struct

```
{
    char sno[10];
    char sname[20];
    int m1,m2;
}student;
```

Declaring a structure variable:

- After defining a structure, we have to declare a variable for the entire structure.
- Once a variable is declared, then the memory will be allocated for the structure members.
- We can declare the structures outside and inside the main() function. If it is declared inside main(), it will have local scope only, whereas if we declare outside main() it will have global scope which allows us to use the structure in other functions also.

Syntax for Tagged structure

Struct typename var1,var2,...;

Syntax for typedef structure:

Typename var1,var2,...;

Ex:

```
struct student s;      (tagged structure)
student s;             (typedef structure)
```

Accessing structure members:

The structure members should be accessed only through a structure variable using a “.”(dot) operator.

Syntax: structurevariable.member

[Type text]

```
/*PROGRAM TO STORE AND READ THE DATA FROM STRUCTURE - tagged
structure*/
#include<stdio.h>
struct student
{
    char id[10];
    char name[15];
    int m1;
    float m2;
    float tot;
};
void main()
{
    struct student s;
    clrscr();
    printf("\nEnter student details\nid name marks1 marks2\n");
    scanf("%s%s%d%f",s.id, s.name, &s.m1, &s.m2);
    s.tot = s.m1 + s.m2;
    printf("\n Hello %s! Your Total marks are: %f",s.name,s.tot);
    getch();
}
```

O/P:

Enter student details

id	name	marks1	marks2
----	------	--------	--------

A123	RAVI	89	98.89
------	------	----	-------

Hello RAVI! Your Total marks are: 187.889999

- We can also declare structure variable by placing them before “;” in structure declaration.

```
struct student
{
    int sno;
    char sname[20];
}s;
```

- We can also initialize only the structure variable as above even without structure tagname.

```
struct
{
    int rno;
    char sname[20];
}s;
```

- But the main disadvantage in removing the structure name is that we can't declare the structure variable separately in any function.

STRUCTURE INITIALIZATION:-

- We can initialize structure variables just like a variable.
- The structure members should not be initialized because; they are not variables, but members of a structure.
- Initialization must be done only to structure variables.

Ex:

```
struct student
{
    int rno;
    char name[20];
};
struct student s={1,"abc"}
```

(or)

```
struct student
{
    int rno;
    char name[20];
}s={1,"abc"};
```

- In structure initialization if some values are missing, then those values will be initialized to zero for numbers or null characters ('\0') for strings.

Ex:

```
struct
{
    int a;
    float b;
    char name[20];
}s1={1,1.1},s2={1};
s1 initializes a to 1, b to 1.1 and name to '\0'.
s2 initializes a to 1, b to 0.0 and name to '\0'.
```

P2

/*PROGRAM TO STORE AND READ THE DATA FROM STRUCTURE - tagged structure*/

```
#include<stdio.h>
```

```
struct student
```

```
{
```

```
    char id[10];
```

```
    char name[15];
```

```
    int m1;
```

```
    float m2;
```

```
    float tot;
```

```
}s2={"A101","RAVI",87,89.78};
```

[Type text]

```
void main()
{
    struct student s1;
    clrscr();
    printf("\n\nEnter student details for s1 \nid name marks1 marks2\n");
    scanf("%s%s%d%f",s1.id, s1.name, &s1.m1, &s1.m2);
    s1.tot = s1.m1 + s1.m2;
    printf("\n\nFrom Structure variable s1 \n Hello %s! Your Total marks are:
                                                    %f",s1.name, s1.tot);

    s2.tot = s2.m1 + s2.m2;
    printf("\nFrom Structure variable s2 ");
    printf("\n\n ID= %s NAME = %s MARKS1 = %d MARKS2= %f TOTAL= %f",
                                                    s2.id, s2.name, s2.m1,s2.m2, s2.tot);

    getch();
}
```

O/P:

Enter student details for s1

id name marks1 marks2

A100 RAVI 88 99.77

From Structure variable s1

Hello RAVI! Your Total marks are: 187.769989

From Structure variable s2

ID= A101 NAME = RAVI MARKS1 = 87 MARKS2= 89.779999 TOTAL= 176.779999

P3.1

/*PROGRAM TO COPY THE VALUES OF STRUCTURE VARIABLES INTO ANOTHR */

```
#include<stdio.h>
```

```
struct student
```

```
{
    char id[10];
    char name[15];
    int m1;
    float m2;
    float tot;
};
```

```
void main()
{
```

```
    struct student s1,s2;
    clrscr();
    printf("\n\nEnter student details\nid name marks1 marks2\n");
    scanf("%s%s%d%f",s1.id, s1.name, &s1.m1, &s1.m2);
```

[Type text]

```
s1.tot = s1.m1 + s1.m2;
s2=s1;
printf("\nHello %s! Your ID is %s Marks1=%d Marks=%f Total marks=%f",
s2.name,s2.id,s2.m1,s2.m2,s2.tot);
getch();
```

```
}
```

O/P:

Enter student details

id name marks1 marks2

A100 RAJ 88 89.786

Hello RAJ! Your ID is A100

Marks1=88

Marks=89.786003

Total marks=177.786011

ARRAY OF STRUCTURES:

We can declare a structure variables as an array to store a set of records.

P4.

/*PROGRAM TO ILLUSTRATE ABOUT ARRAY OF STRUCTURES*/

```
#include<stdio.h>
```

```
struct student
```

```
{
```

```
    char id[10];
```

```
    char name[15];
```

```
    int m1;
```

```
    int m2;
```

```
    int tot;
```

```
};
```

```
void main()
```

```
{
```

```
    struct student s[3];
```

```
    int i;
```

```
    for(i=0;i<3;i++)
```

```
    {
```

```
        printf("\n\nEnter student details for s[%d]",i);
```

```
        printf("\n\nID      NAME MARKS1(int) MARKS2(int)\n\n");
```

```
        scanf("%s%s%d%d",s[i].id, s[i].name, &s[i].m1, &s[i].m2);
```

```
        s[i].tot = s[i].m1 + s[i].m2;
```

```
        printf("\n\nFrom Structure variable s[%d]:",i);
```

```
        printf("\n\n Hello %s! Your Total marks are: %d",s[i].name,s[i].tot);
```

```
        printf("\n\n");
```

```
    }
```

```
    getch();
```

```
}
```

O/P:

Enter student details for s[0]

[Type text]

```
ID      NAME  MARKS1(int) MARKS2(int)
A100   RAJ    88          99
From Structure variable s[0]:
Hello RAJ! Your Total marks are: 187
Enter student details for s[1]
ID      NAME  MARKS1(int) MARKS2(int)
A101    RAM   89          98
From Structure variable s[1]:
Hello RAM! Your Total marks are: 187
Enter student details for s[2]
ID      NAME  MARKS1(int) MARKS2(int)
A102    ROSE  99          98
From Structure variable s[2]:
Hello ROSE! Your Total marks are: 197
```

POINTERS TO STRUCTURES:

We can also store the address of a structure in a pointer.

To access structure members through pointers we have to use the operator -> which is called "**Member selection operator**".

P5

/*PROGRAM TO ILLUSTRATE ABOUT POINTER TO A STRUCTURE*/

```
#include<stdio.h>
```

```
struct student
```

```
{
    char id[10];
    char name[15];
    int m1;
    float m2;
    float tot;
};
```

```
void main()
```

```
{
    struct student s;
    struct student *p;
    clrscr();
    printf("\nEnter student details\nid name marks1 marks2\n");
    scanf("%s%s%d%f",s.id, s.name, &s.m1, &s.m2);
    s.tot = s.m1 + s.m2;
    p=&s;
    printf("\nHello %s! Your Id is %s & Total marks are: %f", p->name, p->id, p->tot);
    getch();
}
```

O/P:

Enter student details

```
id      name marks1 marks2
```

[Type text]

A100 RAJ 88 89.97

Hello RAJ! Your Id is A100 & Total marks are: 177.970001

P6

/*PROGRAM TO ILLUSTRATE ABOUT POINTER TO ARRAY OF STRUCTURES*/

```
#include<stdio.h>
```

```
struct student
```

```
{
```

```
    char name[15];
```

```
    int m1;
```

```
    int m2;
```

```
    int tot;
```

```
};
```

```
void main()
```

```
{
```

```
    int i;
```

```
    struct student s[3] = { "SRUTHI",49,45},{ "SRAVYA",48,47},{ "PRANATHI",49,50} };
```

```
    struct student *p;
```

```
    clrscr();
```

```
    p=s;    /*p=&s gives you warning*/
```

```
    for(i=0;i<3;i++)
```

```
    {
```

```
        p->tot = p->m1 + p->m2;
```

```
        printf("\n\nHello %s! Your Total marks are: %d",p->name, p->tot);
```

```
        p++;
```

```
    }
```

```
    getch();
```

```
}
```

O/P:

Hello SRUTHI! Your Total marks are: 94

Hello SRAVYA! Your Total marks are: 95

Hello PRANATHI! Your Total marks are: 99

NESTED STRUCTURES:

A structure with in a structure is called as "Nested Structure".

Nested structures are used to group some of the members of a structure.

P7

/*PROGRAM TO ILLUSTRATE ABOUT NESTED STRUCTURES*/

```
#include<stdio.h>
```

```
struct student
```

```
{
```

```
    char id[10];
```

```
    char name[15];
```

```
    struct marks
```

```
{
```

[Type text]

```
        int m1;
        float m2;
        float tot;

    }m;
};
void main()
{
    struct student s;
    clrscr();
    printf("\nEnter student details\nid name marks1 marks2\n");
    scanf("%s%s%d%f",s.id, s.name, &s.m.m1, &s.m.m2);
    s.m.tot = s.m.m1 + s.m.m2;
    printf("\nHello %s! Your Total marks are: %f",s.name,s.m.tot);
    getch();
}
```

O/P:

Enter student details

```
id    name  marks1  marks2
A100  RAJ   88        89.787
```

Hello RAJ! Your Total marks are: 177.787003

UNIONS:

- Union is a collection of elements of different datatypes.
- The difference between structures and unions will be in terms of storage.
- Each member of a structure has its own memory location, whereas all the members of the union share the same memory location.

Syntax:

union tagname

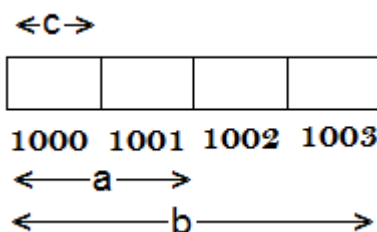
```
{
    data type member1;
    data type member2;
    --
    --
};
```

Ex:

```
union
{
```

```
    int a;
    float b;
    char c;
```

```
}e;
```



[Type text]

- The size of the above union is 4 bytes which is the size of the largest data type.
- We can access only one member at a time because same memory is used by all the members of a union.

P8

/*PROGRAM TO ILLUSTRATE ABOUT UNION*/

```
#include<stdio.h>
```

```
union student
```

```
{
```

```
    char name[15];
```

```
    int m1;
```

```
    float m2;
```

```
};
```

```
void main()
```

```
{
```

```
    union student u;
```

```
    clrscr();
```

```
    strcpy(u.name,"Raj");
```

```
    printf("\nHello %s",u.name);
```

```
    u.m1=88;
```

```
    printf("\nMarks1= %d",u.m1);
```

```
    u.m2=96.57;
```

```
    printf("\nMarks2= %f",u.m2);
```

```
    /*if we assign all at once as follows observe the output...
```

```
    No need to write the below code in the exams*/
```

```
    strcpy(u.name, "Ravi");
```

```
    u.m1=88;
```

```
    u.m2=96.57;
```

```
    printf("\n\n\n Hello %s\n Marks1=%d \n Marks2= %f", u.name, u.m1,u.m2);
```

```
    getch();
```

```
}
```

O/P:

Hello Raj

Marks1= 88

Marks2= 96.570000

Hello #?\$/) @ /*Garbage*/

Marks1=9175 /*Garbage*/

Marks2= 96.570000 /*Last entry will be Correct value*/

STRUCTURES AND FUNCTIONS:

We can pass a structure to function in 3 ways

- Passing individual member of a structure

[Type text]

- Passing entire structure
- Passing the address of a structure (pointers)

1. Passing individual member of a structure:

- We can pass each member of a structure individually to a structure
- In the function these values of members will be stored in ordinary variables
- The problem in this type is if structure contains more no of members, then it will be difficult to pass each member separately in to a function.

P9

/*SENDING INDIVIDUAL ELEMENTS OF A STRUCTURE TO A FUNCTION*/

```
#include<stdio.h>
```

```
struct student
```

```
{
```

```
    char id[10];
```

```
    char name[20];
```

```
    float m;
```

```
};
```

```
show(char[],char[],float);
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    printf("\nEnter id:");
```

```
    gets(s.id);
```

```
    printf("\nEnter Name: ");
```

```
    gets(s.name);
```

```
    printf("\nEnter Marks: ");
```

```
    scanf("%f",&s.m);
```

```
    show(s.id,s.name,s.m);
```

```
    getch();
```

```
}
```

```
show(char sid[10],char sname[20],float score)
```

```
{
```

```
    printf("\nYour details are \n\n ID \t NAME \t MARKS \n\n");
```

```
    printf(" %s %s %f",sid,sname,score);
```

```
}
```

O/P:

Enter id: A100

Enter Name: RAJ

Enter Marks: 89.77

Your details are

ID	NAME	MARKS
----	------	-------

A100	RAJ	89.769997
------	-----	-----------

2. Passing an entire array to the structure:

Instead of passing individual members we can pass the entire structure to a function

P10

/*SENDING WHOLE STRUCTURE VARIABLE TO A FUNCTION*/

[Type text]

```
#include<stdio.h>
struct student
{
    char id[10];
    char name[20];
    float m;
}s;
void main()
{
    clrscr();
    printf("\nEnter id:");
    gets(s.id);
    printf("\nEnter Name: ");
    gets(s.name);
    printf("\nEnter Marks: ");
    scanf("%f",&s.m);
    show(s);
    getch();
}
show(struct student p)
{
    printf("\nYour details are \n\n ID \t NAME \t MARKS \n\n");
    printf(" %s \t %s \t %f",p.id,p.name,p.m);
}
```

O/P:

Enter id:A101

Enter Name: RAVI

Enter Marks: 97.63

Your details are

ID	NAME	MARKS
A101	RAVI	97.629997

**** The problem in this type is duplicate copies of structure will be created which leads to wastage of memory.**

3. Passing the address of a structure instead of variables:

- To avoid duplicate copies of structures we pass the address of a structure instead of values.
- To receive the address the formal arguments must be declared as pointers

P11

/*SENDING THE ADDRESS OF A STRUCTURE VARIABLE TO A FUNCTION*/

```
#include<stdio.h>
struct student
{
    char id[10];
    char name[20];
    float m;
```

[Type text]

```
}s;
void main()
{
    clrscr();
    printf("\nEnter id:");
    gets(s.id);
    printf("\nEnter Name: ");
    gets(s.name);
    printf("\nEnter Marks: ");
    scanf("%f",&s.m);
    show(&s);
    getch();
}
show(struct student *p)
{
    printf("\nYour details are \n\n ID \t NAME \t MARKS \n\n");
    printf(" %s \t %s \t %f",p->id,p->name,p->m);
}
```

O/P:

Enter id:Mahi

Enter Name: A103

Enter Marks: 79.89

Your details are

ID	NAME	MARKS
Mahi	A103	79.889999

Note:

The function declaration must be done only after structure is declared.

SELF REFERENTIAL STRUCTURES

A structure which contains a member that points to the same structure type is called “self-referential structure”.

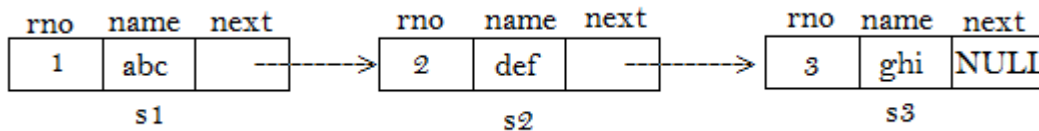
Self-referential structures are used in data structures like “linked list”.

Ex

```
struct student
{
    int rno;
    char name[20];
    struct student *next;
}s1,s2,s3;
main ()
{
    s1.rno=1;
    strcpy (s1.name, "abc");
    s2.rno=2;
    strcpy (s2.name, "def");
```

[Type text]

```
s3.rno=3;
strcpy (s3.name, "ghi");
s1.next=&s2;
s2.next=&s3;
s3.next=NULL;
}
```



/*PROGRAM TO FIND THE ADDITION & MULTIPLICATION OF 2 COMPLEX NUMBERS USING STRUCTURES*/

```
#include <stdio.h>
```

```
typedef struct
```

```
{
```

```
    float real;
```

```
    float imag;
```

```
}complex;
```

```
complex add(complex c1,complex c2);
```

```
complex multiply(complex c1,complex c2);
```

```
int main()
```

```
{
```

```
    complex c1,c2,z,p;
```

```
    printf("For 1st complex number \n");
```

```
    printf("Enter the 1st real number: ");
```

```
    scanf("%f",&c1.real);
```

```
    printf("\nEnter 1st Imaginary number: ");
```

```
    scanf("%f",&c1.imag);
```

```
    printf("\nFor 2nd complex Number\n");
```

```
    printf("Enter the 2nd real number: ");
```

```
    scanf("%f",&c2.real);
```

```
    printf("\nEnter the 2nd imaginary Number: ");
```

```
    scanf("%f",&c2.imag);
```

```
    printf("\nEnter 1st complex number is (%.1f)+i(%.1f)",c1.real,c1.imag);
```

```
    printf("\nEnter 2nd Complex Number is (%.1f)+i(%.1f)\n",c2.real,c2.imag);
```

```
    z=add(c1,c2);
```

```
    printf("Sum of given numbers=(%.1f)+i(%.1f)\n",z.real,z.imag);
```

```
    p=multiply(c1,c2);
```

```
    printf("Multiplication of given numbers=(%.1f)+i(%.1f)",p.real,p.imag);
```

```
}
```

```
complex add(complex c1,complex c2)
```

[Type text]

```
{
    complex z;
    z.real=c1.real+c2.real;
    z.imag=c1.imag+c2.imag;
    return(z);
}
complex multiply(complex c1,complex c2)
{
    complex p;
    p.real = c1.real*c2.real - c1.imag*c2.imag;
    p.imag =c1.imag*c2.real + c1.real*c2.imag;
    return (p);
}
```

[Type text]

Additions

PROGRAM TOWERS OF HONAI

```
#include "stdio.h"
void towers(int, char, char, char);
void towers(int n, char frompeg, char topeg, char auxpeg)
{ /* If only 1 disk, make the move and return */
    if(n==1)
        { printf("\nMove disk 1 from peg %c to peg %c", frompeg, topeg);
          return;
        }
    /* Move top n-1 disks from A to B, using C as auxiliary */
    towers(n-1, frompeg, auxpeg, topeg);
    /* Move remaining disks from A to C */
    printf("\nMove disk %d from peg %c to peg %c", n, frompeg, topeg);
    /* Move n-1 disks from B to C using A as auxiliary */
    towers(n-1, auxpeg, topeg, frompeg);
}
main()
{ int n;
  printf("Enter the number of disks : ");
  scanf("%d", &n);
  printf("The Tower of Hanoi involves the moves :\n\n");
  towers(n, 'A', 'C', 'B');
  return 0;
}
```

O/P:

Enter the number of disks: 4

The Tower of Hanoi involves the moves:

```
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 3 from peg A to peg B
Move disk 1 from peg C to peg A
Move disk 2 from peg C to peg B
Move disk 1 from peg A to peg B
Move disk 4 from peg A to peg C
Move disk 1 from peg B to peg C
Move disk 2 from peg B to peg A
Move disk 1 from peg C to peg A
Move disk 3 from peg B to peg C
Move disk 1 from peg A to peg B
Move disk 2 from peg A to peg C
Move disk 1 from peg B to peg C
```

PROGRAM TO CONVERT THE NUMBER IN WORDS (VERSION 1)

```
/*numwords.c - Converts numbers into words */
#include <stdio.h>
#include <stdlib.h>
char line[80];
/* the line buffer. Stores the numbers as chars */
int i; /* just a for loop counter */
int main(void)
```

[Type text]

```
{
printf("\n\nnumwords\nConverts numbers into words\n");
while(1) {
    printf("Please write a number (or anything else to quit): ");
    fgets(line, sizeof(line), stdin);
    line[strlen(line)-1] = '\0';

    for (i = 0; i < strlen(line); ++i) { /* this for traverses through the character
array that stored the number */
        switch(line[i])
        { /* ... And this switch comes up with each digit as word. */
            case '0': {
                printf("zero ");
                break;
            }
            case '1': {
                printf("one ");
                break;
            }
            case '2': {
                printf("two ");
                break;
            }
            case '3': {
                printf("three ");
                break;
            }
            case '4': {
                printf("four ");
                break;
            }
            case '5': {
                printf("five ");
                break;
            }
            case '6': {
                printf("six ");
                break;
            }
            case '7': {
                printf("seven ");
                break;
            }
            case '8': {
                printf("eight ");
                break;
            }
            case '9': {
                printf("nine ");
                break;
            }
            default: {
                printf("That's not a valid number\n");
                return(0);
            }
        }
    }
}
printf("\n"); /* we want a carriage return after the number completes. */
```

[Type text]

```
/* system("PAUSE"); //this stt is for my reference Good in Windows / Dev-cpp to
keep the console window from closing automatically. Delete it in Linux. */
}
return 0;
}
```

Output:

```
numwords    Converts numbers into words
Please write a number (or anything else to quit): 12345
one two three four five
Please write a number (or anything else to quit): 0890
zero eight nine zero
Please write a number (or anything else to quit): A
That's not a valid number.
```

PROGRAM TO CONVERT THE NUMBER IN WORDS (VERSION 2)

/*Program to Convert Numbers into Words*/

#include<stdio.h>

```
void pw(long, char[]);
char *one[] = {" ", " one", " two", " three", " four", " five", " six", " seven", "eight", "
Nine", " ten", " eleven", " twelve", " thirteen", " fourteen", "fifteen", " sixteen", "
seventeen", " eighteen", " nineteen"};
char *ten[] = {" ", " ", " twenty", " thirty", " forty", " fifty", " sixty", "seventy", "
eighty", " ninety"};
void main()
{
    long n;
    clrscr();
    printf("Enter any number(max 9 digits): ");
    scanf("%9ld", &n);
    if(n <= 0)
        printf("Enter numbers greater than 0");
    else
    {
        pw((n/10000000), "crore");
        pw(((n/100000)%100), "lakh");
        pw(((n/1000)%100), "thousand");
        pw(((n/100)%10), "hundred");
        pw((n%100), " ");
    }
    getch();
}
void pw(long n, char ch[])
{
    (n > 19) ? printf("%s %s", ten[n/10], one[n%10]) : printf("%s ", one[n]);

    if(n)
        printf("%s ", ch);
}
```

Output:

```
Enter any number(max 9 digits): 987654321
ninety eight crore seventy six lakh fifty four thousand three hundred twenty one
```

PROGRAM TO CONVERT INTEGER TO BINARY: /*PROGRAM TO NUMBER TO BINARY */

#include<stdio.h>

#include<conio.h>

[Type text]

```
int main()
{
    int temp,binary[10],count=0,number,index=0;
    printf("Enter a (DECIMAL) number: ");
    scanf("%d",&number);
    printf("\nValue %d in binary ",number);
    while(number>0)
    {
        count++;
        temp=number;
        number=number/2;
        binary[index]=temp%2;
        index++;
    }
    for(index=count-1;index>=0;index--)
    {
        printf(" %d",binary[index]);
    }
    getch();
    return 0;
}
```

OUTPUT:

Enter a (DECIMAL) number: 567

Value 567 in binary: 1 0 0 0 1 1 0 1 1 1

PROGRAM TO CONVERT INTEGER TO ROMAN NUMBER:

```
#include<stdio.h>
#include<conio.h>
int ConvertToRomanNo(int number,int no,char ch);
int main()
{
    int number;
    printf("Enter a DECIMAL Number to convert into ROMAN Number::");
    scanf("%d",&number);
    printf("Roman number of " " %d " "is::",number);
    number=ConvertToRomanNo(number,1000,'M');
    number=ConvertToRomanNo(number,500,'D');
    number=ConvertToRomanNo(number,100,'C');
    number=ConvertToRomanNo(number,50,'L');
    number=ConvertToRomanNo(number,10,'X');
    number=ConvertToRomanNo(number,5,'V');
    number=ConvertToRomanNo(number,1,'I');
    getch();
    return 0;
}

int ConvertToRomanNo(int number,int no,char ch)
{
    int i,j;
    if(number==9)
    {
        printf("IX");
        return (number%9);
    }
    if(number==4)
    {
```

[Type text]

```
        printf("IV");
        return (number%4);
    }

    j=number/no;
    for(i=1;i<=j;i++)
    {
        printf("%c",ch);
    }
    return(number%no);
}
```

Output:

Enter a DECIMAL Number to convert into ROMAN Number::1234
Roman number of 1234 is::MCCXXXIV
PROGRAM TO CONVERT ROMAN NUMERAL TO INTEGER:

/*Code To Convert Roman Numeral To Integer*/

```
#include<stdio.h>
#include<conio.h>
main()
{
    int n=0,i=0,x=0,ax=0,bx=0,cx=0,dx=0,ex=0,
fx=0,gx=0,g[10],a[10],b[10],c[10],d[10],e[10],f[10];
    char r[10];
    printf("please enter the roman numeral :");
    scanf("%s",&r);
    for(;i<10;i++)
    {
        switch(r[i])
        {
            case 'I':
                a[ax]=i; ++ax;
                break;
            case 'V':
                b[bx]=i; ++bx;
                break;
            case 'X':
                c[cx]=i; ++cx;
                break;
            case 'L':
                d[dx]=i; ++dx;
                break;
            case 'C':
                e[ex]=i; ++ex;
                break;
            case 'D':
                f[fx]=i; ++fx;
                break;
            case 'M':
                g[gx]=i; ++gx;
                break;
            default:
                ;
        }
    }
    for(i=0;r[i]!=0 ;i++)
    {
        x++;
    }
}
```

[Type text]

```
n = 1000*(gx);
for(i=0;i<fx;i++)
{
if(gx==0)
n=500*fx;
else if(f[i]>g[gx-1])
n=n+500;
else
n=n-500;
}
for(i=0;i<ex;i++)
{
if(gx==0&&fx==0)
n=100*ex;
else if(fx==0&&gx!=0&&e[i]<g[gx-1])
n=n-100;
else if(fx==0&&gx!=0&&e[i]>g[gx-1])
n=n+100;
else if(fx!=0&&gx==0&&e[i]<f[fx-1])
n=n-100;
else if(fx!=0&&gx==0&&e[i]>f[fx-1])
n=n+100;
else if(fx!=0&&gx!=0&&e[i]>f[fx-1]&&e[i]>g[gx-1])
n=n+100;
else if(fx!=0&&gx!=0&&e[i]>g[gx-1]&&e[i]<f[fx-1])
n=n-100;
}
n= ((50*dx)+n);
n= ((10*cx)+n);
for(i=0;i<cx;i++)
{
if(c[i]<e[ex-1]&&c[i]<d[dx-1])
n = n-20;
}
n= ((5*bx)+n);
n= ((1*ax)+n);
for(i=0;i<ax;i++)
{
if(a[i]<b[bx-1]&&a[i]<c[cx-1])
n = n-2;
}
printf("value is %d ",n);
getch();
return 0;
}
```

/*SOME ERROR IS THERE IN THE PROG*/

OUTPUT:

please enter the roman numeral: MCCXXXIV

value is 1236

(IT IS 1234....NOT 1236)

Left to the aspirant to resolve it

POINTERS:

- A pointer is a variable which stores the address of another variable.
- A pointer is a derived data type in c.
- Pointers contain memory addresses as their values.
- Pointers can be used to access and manipulate data stored in the memory.

Pointers concept is built using the following 3 concepts:

- pointer constant
- pointer value
- pointer variable
- Every memory location has an address which is constant. It is called "**pointer constant**".
- To access the memory address we have to use the "**address of**" operator (&).
- The value thus obtained is called "**pointer value**".
- We can store the address in another variable.
- A variable which stores the address of a pointer value is called as a "**pointer variable**".
- & → address of an operator to know the address of a variable.
- * → value at address operator is used to get the value of the variable there in the pointer.

DECLARATING A POINTER VARIABLE:

Syntax: `datatype *pointer variable;`

- The " * " (asterisk) tells the compiler that the variable is a pointer.
- The datatype is the type of the value, whose address is to be stored.
- Ex: `float a=3.14; float *p=&a;`
Here `float` is not the datatype of the pointer variable `p` but is the datatype of the variable `a`, whose address is stored in that pointer variable `p`.

INITIALIZING A POINTER:

- Assigning address of a variable to a pointer is called "initialization".
- To get the address, we have to use "&" symbol.
Ex: `int a;`
`int *p; //declaration//`
`p=&a; //initialization//`
- A pointer variable will be assigned null if the address is not assigned to it.
- We can declare and initialize pointers as follows.
Ex: `int a;`
`int *p=&a;`
- We can assign a simple character called 'NULL' to a pointer variable.
`p= '\0'` (or) `p=NULL`
- We can assign "0" to a pointer variable.
`p=0;`

ACCESSING THE VALUE IN THE VARIABLE THROUGH THE POINTERS:

- We can access the value of the variable there in the pointer by using *, which is called "indirection operator" or "dereferencing operator" or "value at the address of" operator.

/*PROGRAM TO ILLUSTRATE ABOUT A POINTER*/

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
    float a=4.5;
```

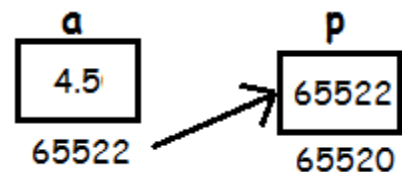
```
    float *p=&a;
```

```
    clrscr();
```

```
    printf("\n\t a= %f &a=%u *(&a)=%f p=%u &p=%u *(&p)=%u *p=%f", a,&a,  
        *(&a), p,&p,*(&p),*p);
```

```
    getch();
```

```
}
```



O/P:

a= 4.500000 &a=65522 *(&a)=4.500000

p=65522 &p=65520 *(&p)= 65522 *p=4.500000

/*PROGRAM TO ADD TWO NO.S USING POINTERS*/

```
#include<stdio.h>
```

```
main()
```

```
{
```

```
    int a=10,b=20;
```

```
    int *pa=&a,*pb=&b;
```

```
    clrscr();
```

```
    printf("Sum=%d", *pa+*pb);
```

```
    getch();
```

```
    return 0;
```

```
}
```

O/P: Sum=30;

ADVANTAGES OF POINTERS:

- Pointers increase the speed of execution.
- Pointers are more efficient in handling arrays.
- Pointers allow C to support dynamic memory management.
- Pointers reduce length and complexity of programs.
- Pointers save storage space in memory.

POINTER EXPRESSIONS:

- Like variables, we can also use pointers in expressions.
Ex: Sum = *p1 + *p2;
 Div = *p1 / *p2;
 Prod = *p1 * *p2;
- We can add or subtract integers using pointers.
Ex: p+2, p-4;
- The value will be incremented with the size of datatype which is called "**scale factor**" i.e., p+2 will be incremented as p+2 * scale factor(size of datatype)
- We can subtract a pointer from another pointer. It gives the no. of elements if they are pointed to an array.
Ex: p2-p1;
- We cannot perform addition, multiplication and division on pointers.
Ex: p1+p2, p1*p2, p2/p1 are **not allowed**

POINTERS - INTER FUNCTION COMMUNICATION:

One of the most useful applications of pointers is functions.

- Passing address
- Functions returning pointers

Passing address: (Call By Reference)

- To pass pointers to a function, we have to specify the address in the function call.
- To receive those addresses we have to declare pointer variables in the function definition.
Ex: Call by Value in Functions

/* PROGRAM TO ILLUSTRATE ABOUT CALL BY REFERENCE*/

```
#include<stdio.h>
void Swap(int *,int *);
main()
{
    int a=10, b=20;
    clrscr();
    printf(" \n\n\t In main, The values before swap: a=%d b=%d", a,b);
    Swap(&a, &b);
    printf(" \n\n\t In main, The values after swap: a=%d b=%d", a,b);
    getch();
}
void Swap(int *pa, int *pb)
{
    int c;
    printf(" \n\n\t in swap x=%u \t y=%u", &pa,&pb);
    c=*pa;
    *pa=*pb;
    *pb=c;
```

[Type text]

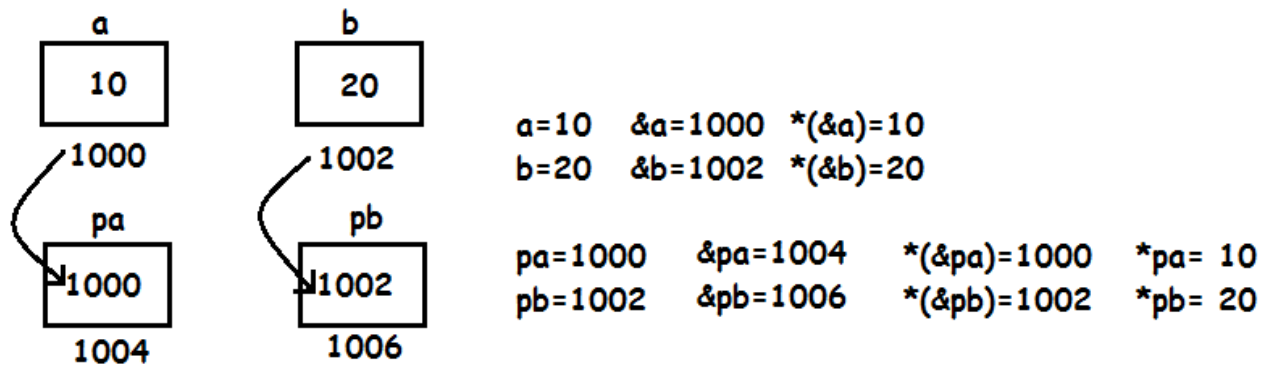
```
printf("\n\t The values after swap in swap function: *pa=%d *pb=%d", *pa,*pb);
}
```

O/P:

In main, The values before swap: a=10 b=20

The values after swap in swap function: *pa=20 *pb=20

In main, The values after swap: a=20 b=10



Function returning pointers:

Observe the following program in which function as pointer which is returning the address of a variable.

/*PROGRAM TO ILLUSTRATE ABOUT FUNCTION RETURNING POINTERS*/

```
int *big(int*, int*);
void main()
{
    int a=10,b=20;
    int*p;
    p=big(&a,&b);
    printf("Bign is %d", *p);
    getch();
}
int *big(int*pa, int*pb)
{
    clrscr();
    if(*pa > *pb)
        return(pa);
    else
        return(pb);
}
```

O/P:

BIG is 20

POINTER TO A POINTER:

We can store the address of a pointer variable in another pointer variable.

- This is called "**Pointer to a Pointer**" concept.
- To declare a pointer to pointer variable we have to use two indirection operators before the pointer variable.

/*PROGRAM TO ILLUSTRATE ABOUT POINTER TO A POINTER*/

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a=6;
    int *pa=&a;
    int **ppa=&pa;
    clrscr();

    printf("\na=%d &a=%u *(&a)=%u",a,&a,*(&a));
    printf("\npa=%u &pa=%u *(&pa)=%u *pa=%d", pa,&pa,*(&pa), *pa);
    printf("\nppa=%u &ppa=%u *(&ppa)=%u *ppa=%u **ppa=%d", ppa, &ppa, *(&ppa), *ppa,
**ppa);
    getch();
}
```

O/P:

```
a=6 &a=65492 *(&a)=6
pa=65492 &pa=65494 *(&pa)=65492 *pa=6
ppa=65494 &ppa=65496 *(&ppa)=65494 *ppa=65492 **ppa=6
```

COMPATIBILITY:

Compatibility means assigning the address of variable of some data type to a pointer of same type.

We cannot assign one type of address to another type of pointer.

Compatibility should be considered in the following three cases

1. Pointer size.
2. Dereference type.
3. Dereference level.

1. Pointer size compatibility:

The pointer size is same for memory addresses which is 2 of bytes (size of a memory address of a computer).

The size of all variables to which a pointer is referring will be different.

/*PROGRAM TO ILLUSTRATE ABOUT POINTER SIZE COMPATIBILITY*/

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a=5,*pa=&a;
```

[Type text]

```
float b=6, *pb=&b;
char c='c', *pc=&c;
clrscr();
printf("\nSIZE OF a=%d int=%d pa=%d",sizeof(a),sizeof(int),sizeof(pa));
printf("\nSIZE OF b=%d float=%d pb=%d",sizeof(b),sizeof(float), sizeof(pb));
printf("\nSIZE OF c=%d char=%d pc=%d",sizeof(c),sizeof(char),sizeof(pc));
getch();
}
```

O/P:

SIZE OF a=2 int =2 pa=2

SIZE OF b=4 float=4 pb=2

SIZE OF c=1 char=1 pc=2

In practice, pointers will be size 2 on 16 bit machine 4 on 32-bit machine, 8 on 64 bit machine.

2. De-reference type compatibility:

The dereference type is data type of a variable to which a pointer is referencing.

Except in one case, we cannot assign the address of type into another type of pointer.

EX: int a;
char *p;

p = &a; → invalid because of different types.

The incompatibility can be solved by typecasting.

EX: int a;
char *p;
p = (char*)&a;

/*PROGRAM TO ILLUSTRATE ABOUT THE POINTER TYPE COMPATIBILITY*/

```
void main()
{
    int a=65;
    char *p;
    clrscr();
    p = (char*)&a;
    printf("\na=%d a=%c *p=%d *p=%c",a,a,*p,*p);
    printf("\n*p+1=%d *p+1=%c",*p+1,*p+1);

    getch();
}
```

O/P:

a=65 a=A *p=65 *p=A

***p+1=66 *p+1=B**

NOTE:

THIS TYPE COMAPTIBILITY WON'T WORKS FOR TYPECASTING THE OTHER CONVERSIONS\

LIKE FLOAT → INT AND FLOAT → CHAR

[Type text]

Pointer void:

- The special case where we can directly assign one type of address into another type of Pointer is in the concept of void pointers.
- A void pointer is a generic pointer which can hold the address of any type of variable.

EX: void *p;

/*PROGRAM TO ILLUSTRATE ABOUT THE POINTER TYPE COMPATIBILITY VOID*/

```
void main()
{
    int a=65;
    float b=7.8;
    char ch='A';
    void *p;
    clrscr();
    p = &a;
    printf("\na=%d",*(int *)p);
    p=&b;
    printf("\nb=%f",*(float *)p);
    p=&ch;
    printf("\nch=%c",*(char *)p);

    getch();
}
```

O/P:

a=65

b=7.800000

ch=A

Dereference level compatibility:

Compatibility should be compared in pointer level also.

A pointer to int is not compatible with a pointer-to-pointer to int has a reference type of pointer to int.

EX: int a;

int *pa;

int **ppa; but *ppa=&a is invalid.

pa=&a;

ppa=&pa;

POINTERS AND ARRAYS:

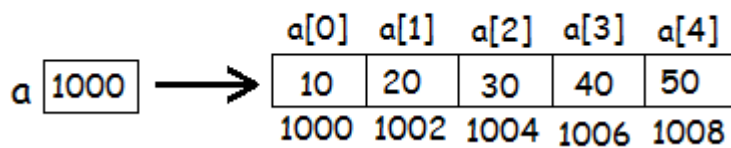
The array name is by a default a pointer containing the address of first element.

EX: int a[5]={10,20,30,40,50}

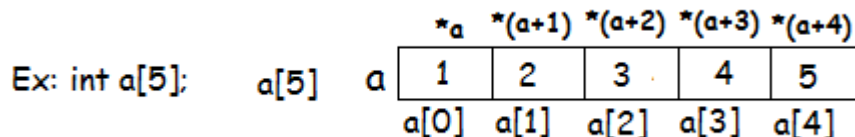
The memory allocations for this array are

[Type text]

Ex: `int a[5] = { 10, 20, 30, 40, 50};`



- We can print the first element in an array using index or a pointer.
EX: `printf("%d%d", a[0], *a);`
o/p: 10 10
- `a[0]` and `*a` represents the value in the first memory location i.e., 10
- To access the next element we have to increase the variable "a"
Ex: `a+1` represents the address of second element.
`a+2` represents the address of third element.
`a+n` can be calculated using the formulae `a + n * (size of the element)`.
- We can increment or decrement a pointer variable to access the values in an array.
- For example if a pointer variable `p` contains an address of a second element then `p--` will move the pointer to the first element.
- We can access these values using the indirection operator "`*`".



/* PROGRAM TO ILLUSTRATE ABOUT ARRAY AS POINTER */

```
void main()
{
    int a[3]={6,7,8},i;
    clrscr();
    for(i=0;i<3;i++)
        printf("\na[%d]=%d  a[%d]=%d a[%d]=%d",i,a[i],i,*(a+i),i,*a+i);
    getch();
}
```

O/P:

```
a[0]=6  a[0]=6 a[0]=6
a[1]=7  a[1]=7 a[1]=7
a[2]=8  a[2]=8 a[2]=8
```

/*PROGRAM TO ILLUSTRATE ABOUT POINTER TO 1DIM ARRAY*/

```
void main()
{
    int a[3]={6,7,8},i;
    int *p;
```

[Type text]

```
p=a; /* or we can write p=&a[0]; */
clrscr();
for(i=0;i<3;i++)
    printf("\na[%d]=%d a[%d]=%d a[%d]=%d",i,a[i],i,*a+i,i,*a+i);

printf("\n\n");

for(i=0;i<3;i++)
    printf("\na[%d]=%d a[%d]=%d a[%d]=%d",i,a[i],i,*(p+i),i,*p+i);
getch();
}
```

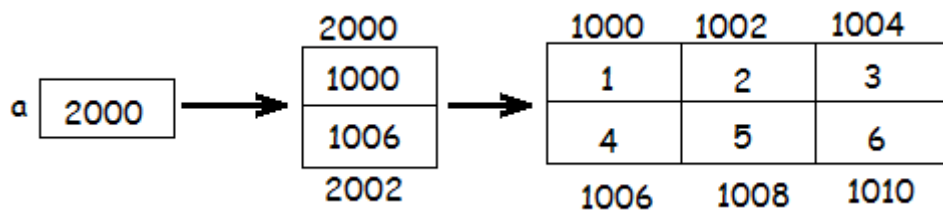
O/P:

a[0]=6 a[0]=6 a[0]=6
a[1]=7 a[1]=7 a[1]=7
a[2]=8 a[2]=8 a[2]=8

a[0]=6 a[0]=6 a[0]=6
a[1]=7 a[1]=7 a[1]=7
a[2]=8 a[2]=8 a[2]=8

POINTERS – 2DIM ARRAYS

- We can use 2-D Arrays with pointer Notation
- To display an element in a 2-D Array using pointers the notation is $*(a+i)+j$
- The memory for a 2-D array will be created continuously for all elements.
Ex: `int a[2][3]={{1,2,3},{4,5,6}}`
- The memory allocation will be



Ex: Let us find out the value of `a[1][2]` = 6

$*(a+i)+j$ = $*(\text{value at}(\text{base address of } a + \text{index value} * \text{scale factor}))$
 $*(a+1)+2$ = $*(\text{value at}(2000 + 1 * 2)+2)$ ($i=1$ $j=2$)
= $*(\text{value at } (2002)+2)$
= $*(1006 + 2 * \text{scale factor})$
= $\text{value at } (1006 + 4)$
= $\text{value at } (1010)$
= 6

/* PROGRAM TO ILLUSTRATE ABOUT THE POINTERS TO 2DIM ARRAYS*/

```
main()
{
    int a[2][3]={{1,2,3},{4,5,6}},i,j;
    int **p;
```

[Type text]

```
clrscr();

for(i=0;i<2;i++)
{
    for(j=0;j<3;j++)
    {
        printf("a[%d][%d]=%d",i,j,*(*(a+i)+j));

    }
    printf("\n");
}

printf("\n\n");
for(i=0;i<2;i++)
{
    for(j=0;j<3;j++)
    {
        printf("\n\tAddress a[%d][%d]=%u",i,j,&a[i][j]);

    }
}
printf("\n\n\n");

p='\0';
for(i=0;i<2;i++)
{
    p[i]= &a[i][0];
    for(j=0;j<3;j++)
    {
        printf("\n\tAddress of a[%d][%d]=%u",i,j,p[i]+j);

    }
}
printf("\n\n");

for(i=0;i<2;i++)
{
    for(j=0;j<3;j++)
    {
        printf("\ta[%d][%d]=%d",i,j,*(*(p+i)+j));

    }
    printf("\n");
}
getch();
}
```

[Type text]

a[0][0]=1 a[0][1]=2 a[0][2]=3
a[1][0]=4 a[1][1]=5 a[1][2]=6

Address a[0][0]=65480
Address a[0][1]=65482
Address a[0][2]=65484
Address a[1][0]=65486
Address a[1][1]=65488
Address a[1][2]=65490

Address of a[0][0]=65480
Address of a[0][1]=65482
Address of a[0][2]=65484
Address of a[1][0]=65486
Address of a[1][1]=65488
Address of a[1][2]=65490

a[0][0]=1	a[0][1]=2	a[0][2]=3
a[1][0]=4	a[1][1]=5	a[1][2]=6

POINTERS TO FUNCTIONS:

- Like variables a function has an address in memory
- We can store the address in a pointer variable and using that variable we call the function
- Useful in call back functions, function call at runtime, sorting and search functions
Syntax: datatype (*ptr)();
- The datatype is the returntype of the function
- After declaring a variable we have to assign the name to the pointer variable

/* PROGRAM TO ILLUSTRATE ABOUT THE PTR TO FUN*/

```
void show()
{
    printf("\nshow");
}
void main()
{
    void (*p)();
    clrscr();
    p=show;
    (*p)();
    getch();
}
```

O/P: Show

=====

/* PROGRAM TO ILLUSTRATE ABOUT THE PTR TO FUN WITH ARGS*/

[Type text]

```
int add(int x, int y)
{
    return x+y;
}
void main()
{
    int (*p)();
    int a=5,b=6;
    clrscr();
    p=add;
    printf("\nsum=%d",(*p)(a,b));
    getch();
}
```

O/P: sum=11